

2.6.1 Single Location Descriptions

A single location description is either:

1. A simple location description, representing an object which exists in one contiguous piece at the given location, or
2. A composite location description consisting of one or more simple location descriptions, each of which is followed by one composition operation. Each simple location description describes the location of one piece of the object; each composition operation describes which part of the object is located there. Each simple location description that is a DWARF expression is evaluated independently of any others.

2.6.1.1 Simple Location Descriptions

A simple location description consists of one contiguous piece or all of an object or value.

2.6.1.1.1 Empty Location Descriptions

An empty location description consists of a DWARF expression containing no operations. It represents a piece or all of an object that is present in the source but not in the object code (perhaps due to optimization).

2.6.1.1.2 Memory Location Descriptions

A memory location description consists of a non-empty DWARF expression (see Section 2.5 on page 26), whose value is the address of a piece or all of an object or other entity in memory.

2.6.1.1.3 Register Location Descriptions

A register location description consists of a register name operation, which represents a piece or all of an object located in a given register.

Register location descriptions describe an object (or a piece of an object) that resides in a register, while the opcodes listed in Section 2.5.1.2 on page 28 are used to describe an object (or a piece of an object) that is located in memory at an address that is contained in a register (possibly offset by some constant). A register location description must stand alone as the entire description of an object or a piece of an object.

Chapter 2. General Description

The following DWARF operations can be used to specify a register location.

Note that the register number represents a DWARF specific mapping of numbers onto the actual registers of a given architecture. The mapping should be chosen to gain optimal density and should be shared by all users of a given architecture. It is recommended that this mapping be defined by the ABI authoring committee for each architecture.

1. **DW_OP_reg0, DW_OP_reg1, ..., DW_OP_reg31**

The **DW_OP_reg<n>** operations encode the names of up to 32 registers, numbered from 0 through 31, inclusive. The object addressed is in register *n*.

2. **DW_OP_regx**

The **DW_OP_regx** operation has a single unsigned LEB128 literal operand that encodes the name of a register.

These operations name a register location. To fetch the contents of a register, it is necessary to use one of the register based addressing operations, such as [DW_OP_bregx](#) (Section 2.5.1.2 on page 28).

2.6.1.1.4 Implicit Location Descriptions

An implicit location description represents a piece or all of an object which has no actual location but whose contents are nonetheless either known or known to be undefined.

The following DWARF operations may be used to specify a value that has no location in the program but is a known constant or is computed from other locations and values in the program.

1. **DW_OP_implicit_value**

The **DW_OP_implicit_value** operation specifies an immediate value using two operands: an unsigned LEB128 length, followed by a sequence of bytes of the given length that contain the value.

2. **DW_OP_stack_value**

The **DW_OP_stack_value** operation specifies that the object does not exist in memory but its value is nonetheless known and is at the top of the DWARF expression stack. In this form of location description, the DWARF expression represents the actual value of the object, rather than its location. The **DW_OP_stack_value** operation terminates the expression.

3. **DW_OP_implicit_pointer**

An optimizing compiler may eliminate a pointer, while still retaining the value that the pointer addressed. DW_OP_implicit_pointer allows a producer to describe this value.

The DW_OP_implicit_pointer operation specifies that the object is a pointer that cannot be represented as a real pointer, even though the value it would point to can be described. In this form of location description, the DWARF expression refers to a debugging information entry that represents the actual value of the object to which the pointer would point. Thus, a consumer of the debug information would be able to show the value of the dereferenced pointer, even when it cannot show the value of the pointer itself.

The DW_OP_implicit_pointer operation has two operands: a reference to a debugging information entry that describes the dereferenced object's value, and a signed number that is treated as a byte offset from the start of that value. The first operand is a 4-byte unsigned value in the 32-bit DWARF format, or an 8-byte unsigned value in the 64-bit DWARF format (see Section 7.4 on page 196). The second operand is a signed LEB128 number.

The first operand is used as the offset of a debugging information entry in a .debug_info section, which may be contained in an executable or shared object file other than that containing the operator. For references from one executable or shared object file to another, the relocation must be performed by the consumer.

The debugging information entry referenced by a DW_OP_implicit_pointer operation is typically a DW_TAG_variable or DW_TAG_formal_parameter entry whose DW_AT_location attribute gives a second DWARF expression or a location list that describes the value of the object, but the referenced entry may be any entry that contains a DW_AT_location or DW_AT_const_value attribute (for example, DW_TAG_dwarf_procedure). By using the second DWARF expression, a consumer can reconstruct the value of the object when asked to dereference the pointer described by the original DWARF expression containing the DW_OP_implicit_pointer operation.

DWARF location descriptions are intended to yield the **location** of a value rather than the value itself. An optimizing compiler may perform a number of code transformations where it becomes impossible to give a location for a value, but it remains possible to describe the value itself. Section 2.6.1.1.3 on page 39 describes operators that can be used to describe the location of a value when that value exists in a register but not in memory. The operations in this section are used to describe values that exist neither in memory nor in a single register.

2.6.1.2 Composite Location Descriptions

A composite location description describes an object or value which may be contained in part of a register or stored in more than one location. Each piece is described by a composition operation, which does not compute a value nor store any result on the DWARF stack. There may be one or more composition operations in a single composite location description. A series of such operations describes the parts of a value in memory address order.

Each composition operation is immediately preceded by a simple location description which describes the location where part of the resultant value is contained.

1. **DW_OP_piece**

The DW_OP_piece operation takes a single operand, which is an unsigned LEB128 number. The number describes the size in bytes of the piece of the object referenced by the preceding simple location description. If the piece is located in a register, but does not occupy the entire register, the placement of the piece within that register is defined by the ABI.

Many compilers store a single variable in sets of registers, or store a variable partially in memory and partially in registers. DW_OP_piece provides a way of describing how large a part of a variable a particular DWARF location description refers to.

2. **DW_OP_bit_piece**

The DW_OP_bit_piece operation takes two operands. The first is an unsigned LEB128 number that gives the size in bits of the piece. The second is an unsigned LEB128 number that gives the offset in bits from the location defined by the preceding DWARF location description.

Interpretation of the offset depends on the location description. If the location description is empty, the offset doesn't matter and the DW_OP_bit_piece operation describes a piece consisting of the given number of bits whose values are undefined. If the location is a register, the offset is from the least significant bit end of the register. If the location is a memory address, the DW_OP_bit_piece operation describes a sequence of bits relative to the location whose address is on the top of the DWARF stack using the bit numbering and direction conventions that are appropriate to the current language on the target system. If the location is any implicit value or stack value, the DW_OP_bit_piece operation describes a sequence of bits using the least significant bits of that value.

DW_OP_bit_piece is used instead of DW_OP_piece when the piece to be assembled into a value or assigned to is not byte-sized or is not at the start of a register or addressable unit of memory.

D.13 Implicit Pointer Examples

If the compiler determines that the value of an object is constant (either throughout the program, or within a specific range), it may choose to materialize that constant only when used, rather than store it in memory or in a register. The [DW_OP_implicit_value](#) operation can be used to describe such a value.

Sometimes, the value may not be constant, but still can be easily rematerialized when needed. A DWARF expression terminating in [DW_OP_stack_value](#) can be used for this case. The compiler may also eliminate a pointer value where the target of the pointer resides in memory, and the [DW_OP_stack_value](#) operator may be used to rematerialize that pointer value. In other cases, the compiler will eliminate a pointer to an object that itself needs to be materialized. Since the location of such an object cannot be represented as a memory address, a DWARF expression cannot give either the location or the actual value or a pointer variable that would refer to that object. The [DW_OP_implicit_pointer](#) operation can be used to describe the pointer, and the debugging information entry to which its first operand refers describes the value of the dereferenced object. A DWARF consumer will not be able to show the location or the value of the pointer variable, but it will be able to show the value of the dereferenced pointer.

Consider the C source shown in Figure D.58. Assume that the function `foo` is not inlined, that the argument `x` is passed in register 5, and that the function `foo` is optimized by the compiler into just an increment of the volatile variable `v`. Given these assumptions a possible DWARF description is shown in Figure D.59 on the following page.

```
struct S { short a; char b, c; };
volatile int v;
void foo (int x)
{
    struct S s = { x, x + 2, x + 3 };
    char *p = &s.b;
    s.a++;
    v++;
}
int main ()
{
    foo (v+1);
    return 0;
}
```

Figure D.58: C implicit pointer example #1: source

Appendix D. Examples (Informative)

```
1$: DW_TAG_structure_type
    DW_AT_name("S")
    DW_AT_byte_size(4)
10$: DW_TAG_member
    DW_AT_name("a")
    DW_AT_type(reference to "short int")
    DW_AT_data_member_location(constant 0)
11$: DW_TAG_member
    DW_AT_name("b")
    DW_AT_type(reference to "char")
    DW_AT_data_member_location(constant 2)
12$: DW_TAG_member
    DW_AT_name("c")
    DW_AT_type(reference to "char")
    DW_AT_data_member_location(constant 3)
2$: DW_TAG_subprogram
    DW_AT_name("foo")
20$: DW_TAG_formal_parameter
    DW_AT_name("x")
    DW_AT_type(reference to "int")
    DW_AT_location(DW_OP_reg5)
21$: DW_TAG_variable
    DW_AT_name("s")
    DW_AT_type(reference to S at 1$)
    DW_AT_location(expression=
        DW_OP_breg5(1) DW_OP_stack_value DW_OP_piece(2)
        DW_OP_breg5(2) DW_OP_stack_value DW_OP_piece(1)
        DW_OP_breg5(3) DW_OP_stack_value DW_OP_piece(1))
22$: DW_TAG_variable
    DW_AT_name("p")
    DW_AT_type(reference to "char *")
    DW_AT_location(expression=
        DW_OP_implicit_pointer(reference to 21$, 2))
```

Figure D.59: C implicit pointer example #1: DWARF description

1 In Figure D.59, even though variables `s` and `p` are both optimized away
2 completely, this DWARF description still allows a debugger to print the value of
3 the variable `s`, namely (2, 3, 4). Similarly, because the variable `s` does not live
4 in memory, there is nothing to print for the value of `p`, but the debugger should
5 still be able to show that `p[0]` is 3, `p[1]` is 4, `p[-1]` is 0 and `p[-2]` is 2.

Appendix D. Examples (Informative)

1 As a further example, consider the C source shown in Figure D.60. Make the
2 following assumptions about how the code is compiled:

- 3 • The function `foo` is inlined into function `main`
- 4 • The body of the `main` function is optimized to just three blocks of
5 instructions which each increment the volatile variable `v`, followed by a
6 block of instructions to return 0 from the function
- 7 • Label `label0` is at the start of the `main` function, `label1` follows the first `v++`
8 block, `label2` follows the second `v++` block and `label3` is at the end of the
9 `main` function
- 10 • Variable `b` is optimized away completely, as it isn't used
- 11 • The string literal `"opq"` is optimized away as well

12 Given these assumptions a possible DWARF description is shown in Figure D.61
13 on the next page.

```
static const char *b = "opq";
volatile int v;
static inline void foo (int *p)
{
    (*p)++;
    v++;
    p++;
    (*p)++;
    v++;
}

int main ()
{
label0:
    int a[2] = 1, 2 ;
    v++;
label1:
    foo (a);
label2:
    return a[0] + a[1] - 5;
label3:
}
```

Figure D.60: C implicit pointer example #2: source

Appendix D. Examples (Informative)

```
1$: DW_TAG_variable
    DW_AT_name("b")
    DW_AT_type(reference to "const char *")
    DW_AT_location(expression=
        DW_OP_implicit_pointer(reference to 2$, 0))
2$: DW_TAG_dwarf_procedure
    DW_AT_location(expression=
        DW_OP_implicit_value(4, {'o', 'p', 'q', '\0'}))
3$: DW_TAG_subprogram
    DW_AT_name("foo")
    DW_AT_inline(DW_INL_declared_inlined)
30$: DW_TAG_formal_parameter
    DW_AT_name("p")
    DW_AT_type(reference to "int *")
4$: DW_TAG_subprogram
    DW_AT_name("main")
40$: DW_TAG_variable
    DW_AT_name("a")
    DW_AT_type(reference to "int[2]")
    DW_AT_location(location list 98$)
41$: DW_TAG_inlined_subroutine
    DW_AT_abstract_origin(reference to 3$)
42$: DW_TAG_formal_parameter
    DW_AT_abstract_origin(reference to 30$)
    DW_AT_location(location list 99$)

! .debug_loclists section
98$: DW_LLE_start_end[<label0 in main> .. <label1 in main>)
    DW_OP_lit1 DW_OP_stack_value DW_OP_piece(4)
    DW_OP_lit2 DW_OP_stack_value DW_OP_piece(4)
    DW_LLE_start_end[<label1 in main> .. <label2 in main>)
    DW_OP_lit2 DW_OP_stack_value DW_OP_piece(4)
    DW_OP_lit2 DW_OP_stack_value DW_OP_piece(4)
    DW_LLE_start_end[<label2 in main> .. <label3 in main>)
    DW_OP_lit2 DW_OP_stack_value DW_OP_piece(4)
    DW_OP_lit3 DW_OP_stack_value DW_OP_piece(4)
    DW_LLE_end_of_list
99$: DW_LLE_start_end[<label1 in main> .. <label2 in main>)
    DW_OP_implicit_pointer(reference to 40$, 0)
    DW_LLE_start_end[<label2 in main> .. <label3 in main>)
    DW_OP_implicit_pointer(reference to 40$, 4)
    DW_LLE_end_of_list
```

Figure D.61: C implicit pointer example #2: DWARF description